



DoS/DDoS - Amplification and Reflection Attacks and Defense

Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks remain as common threat in the Internet. Refer to the report by Cloudflare on DDoS Attack Trends for [2025 Q1](#). You may be interested to also read [A Cisco Guide to Defending Against Distributed Denial of Service Attacks](#).

Broadly, DoS/DDoS may be classified into 2 categories as shown in Figure 1.

DoS/DDoS categories	Description	Typical attack packet rate	Examples
Overwhelming Quantity of Traffic	Threat actor sends an enormous quantity of data at a rate that the network, host, or application cannot handle, possibly causing slow response times or even crash a device or service.	High	ICMP flood, UDP flood, smurf attack, TCP SYN, etc
Maliciously Crafted Packets	Threat actor sends maliciously crafted packets that the host or application is not expected to handle, possibly causing it to run very slowly or even crash.	Low	Slowloris, ping of death, etc

Figure 1

Under category 1 employing volumetric amount of network traffic to overwhelm a target, a typical attack vector is to employ a compromised set of hosts forming a botnet to directly flood the target as shown in Figure 2.

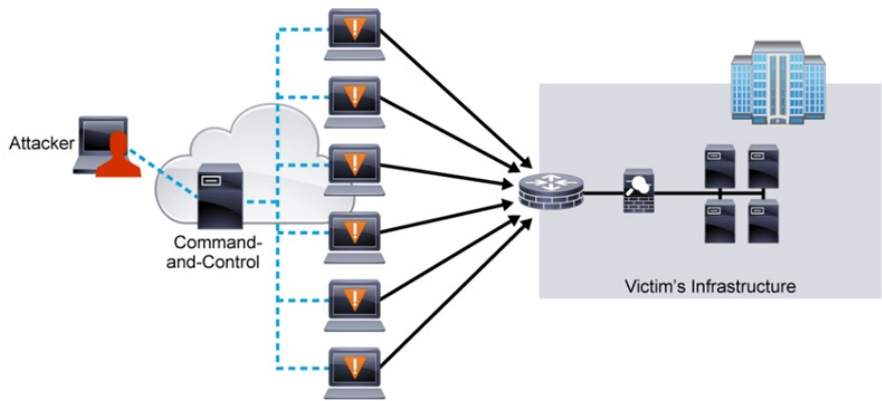


Figure 2

Alternatively even without a botnet, an attacker may still launch volumetric flood on a target by employing the technique of amplification and reflection attack as shown in Figure 3.

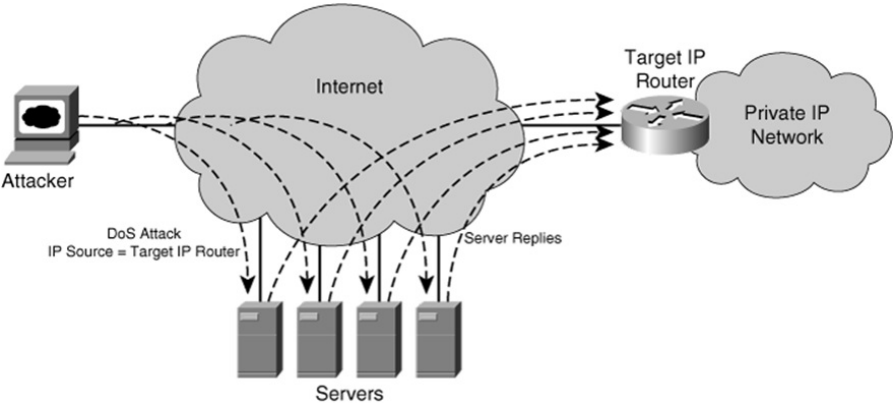


Figure 3

Today, UDP-based servers are especially vulnerable to be misused by an attacker to launch amplification and reflection attack, especially NTP and DNS which are widely available in the Internet. You may be interested to read [Deep Inside a](#)

Amplification and Reflection - Smurf Attack

To have a better understanding of amplification and reflection attack, but to avoid getting into trouble for misusing DNS or NTP servers in the Internet, we will experiment with **Smurf attack** in the lab which was the notorious **amplification and reflection attack** encountered in the Internet in 1990s.

In **Smurf attack**, a single **broadcast ping** is sent to a **directed broadcast address** by an attacker using a **spoofed source IP address** belonging to a target which would result in multiple ping replies (each from an intermediary in the directed broadcast subnet), to be reflected to the target as shown in Figure 4.

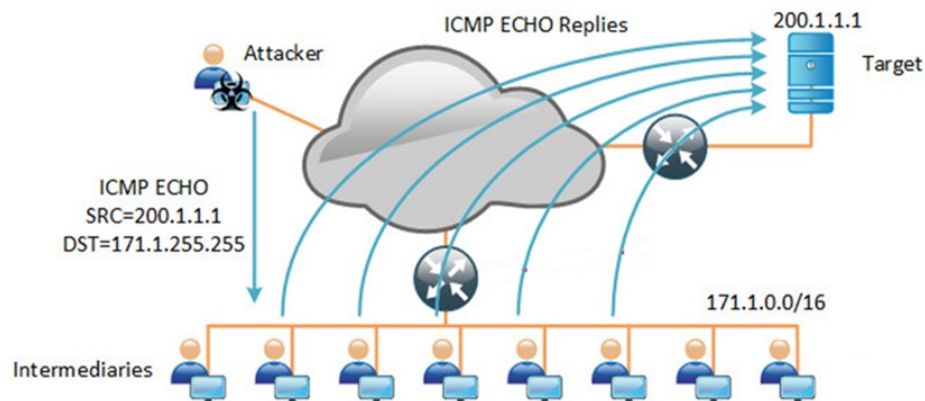


Figure 4

In Kali Linux, the **hping3** tool as shown in Figure 5 can be used to conduct **Smurf attack** which you will try out in the lab exercise.

```
(kali@kali)-[~]
└─$ hping3 --help
usage: hping3 host [options]
  -h --help          show this help
  -v --version       show version
  -c --count         packet count
  -i --interval      wait (uX for X microseconds, for example -i u1000)
  -f --fast          alias for -i u10000 (10 packets for second)
  -F --faster        alias for -i u1000 (100 packets for second)
  -flood            sent packets as fast as possible. Don't show replies.
  -n --numeric       numeric output
  -q --quiet         quiet
  -I --interface     interface name (otherwise default routing interface)
  -V --verbose       verbose mode
  -D --debug         debugging info
  -z --bind          bind ctrl+z to ttl (default to dst port)
  -Z --unbind       unbind ctrl+z
  -beep            beep for every matching packet received

Mode
  default mode      TCP
  -0 --rawip        RAW IP mode
  -1 --icmp         ICMP mode
  -2 --udp          UDP mode
  -8 --scan         SCAN mode.
                    Example: hping --scan 1-30,70-90 -S www.target.host
  -9 --listen       listen mode

IP
  -a --spoof        spoof source address
```

Figure 5

Defending against Amplification and Reflection Attacks

Today, **Smurf attack** is largely defended by **disabling** support for **directed broadcast** in routers and **disabling** response to **broadcast ping** in OS. Nevertheless, there is still a need to be aware of **UDP-based amplification and reflection attacks** which are still feasible and possible to be encountered in today's Internet.

The key characteristic of **amplification and reflection attacks** is an attacker is able to **spoof** the **source IP address** belonging to the target. Hence, a defense is to adopt **RFC 2827** also known as **BCP 38** to prevent source IP address spoofing.

Specifically, since ISPs are aware of the public IP address blocks allocated to their customers, BCP 38 recommends ISPs to filter packets with source IP addresses not matching their allocated IP addresses. An example is shown in Figure 6.

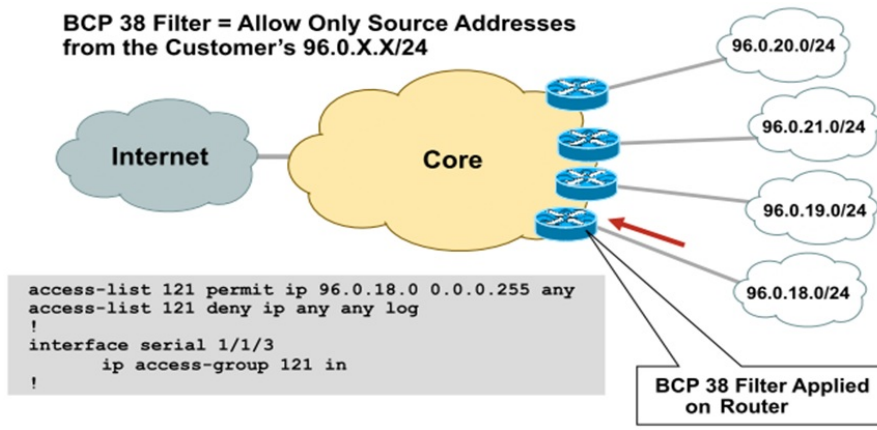


Figure 6

In addition, enterprise should also [filter](#) at their [edge routers](#) those packets with source IP addresses not matching its allocated public IP addresses. In fact, filtering can also be applied at the [distribution switches](#) to deny packets with source IP addresses not matching the subnet address block. An example is shown in Figure 7.

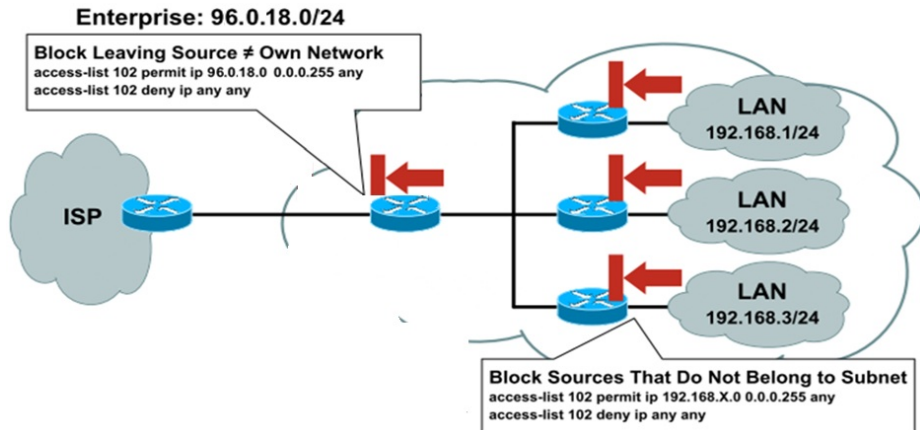


Figure 7

Unfortunately it's been 20+ years since the first publication of RFC 2827/BCP 38, not everyone implements the recommendations. As a result, packets with spoofed IP addresses continue to plague the Internet.

So, it's beneficial for enterprise to implement [anti-spoofing ACL](#) at its [edge routers](#) to deny packets with clearly-known spoofed source IP addresses from entering its network. An example is shown in Figure 8.

!--- Anti-spoofing entries are shown here.

!--- Deny RFC 3330 special-use address

```

access-list 110 deny ip host 0.0.0.0 any
access-list 110 deny ip 127.0.0.0 0.255.255.255 any
access-list 110 deny ip 192.0.2.0 0.0.0.255 any
access-list 110 deny ip 224.0.0.0 31.255.255.255 any

```

!--- Filter RFC 1918 space.

```

access-list 110 deny ip 10.0.0.0 0.255.255.255 any
access-list 110 deny ip 172.16.0.0 0.15.255.255 any
access-list 110 deny ip 192.168.0.0 0.0.255.255 any

```

!--- Deny your space as source from entering your AS. !--- Deploy only at the AS edge.

```

access-list 110 deny ip YOUR_CIDR_BLOCK any

```

!--- Deny access to internal infrastructure addresses.

```

access-list 110 deny ip any INTERNAL_INFRASTRUCTURE_ADDRESSES

```

!--- Permit transit traffic.

```

access-list 110 permit ip any any

```

Figure 8